HTML + CSS + JS

সম্পূর্ণ Web Development গাইড — Zero থেকে Advanced

ওয়েবসাইট কখনো বানাওনি? কোনো সমস্যা নেই। এই গাইড তোমাকে HTML দিয়ে কাঠামো, CSS দিয়ে সাজানো এবং JavaScript দিয়ে প্রাণ দেওয়া — সবটা শেখাবে একদম শুরু থেকে।

🌐 HTML — কাঠামো | 🎨 CSS — ডিজাইন | ⚡ JS — কার্যকারিতা | 📖 14 Chapters | 💡 100+ Examples | 🌙 Dark Mode

https://mywebsite.com

```
<h1>আমার প্রথম ওয়েবসাইট</h1>
h1 { color: coral; }
document.querySelector('h1').addEventListener('click',
() ⇒ alert('Hello!'))
```

📋 সূচিপত্র

🌐 PART 1 — HTML

01 HTML পরিচিতি ও ওয়েবের কাঠামো Ch. 1

HTML কী, Tag, Element, Attribute — ওয়েবের হাড়গোড়

02 গুরুত্বপূর্ণ HTML Tags Ch. 2

Heading, Paragraph, Link, Image, List, Table, Form

03 Semantic HTML ও Forms Ch. 3

header, nav, main, section, footer, form input types

🎨 PART 2 — CSS

04 CSS পরিচিতি ও Selectors Ch. 4

CSS কী, Selectors, Properties, Box Model

05 Typography, Colors ও Units Ch. 5

Font, Color, px/em/rem/%, Gradient, CSS Variables

06 Flexbox — আধুনিক Layout Ch. 6

flex-direction, justify-content, align-items, flex-wrap

07 CSS Grid ও Responsive Design Ch. 7

Grid template, Media Queries, Mobile-first design

⚡ PART 3 — JAVASCRIPT

08 JavaScript পরিচিতি ও Basics Ch. 8

Variables, Data Types, Operators, Conditions, Loops

09 Functions ও Arrays **Ch. 9**

Function declaration, Arrow function, Array methods

10 DOM Manipulation **Ch. 10**

querySelector, innerHTML, Events, classList — Page বদলানো

11 Objects, JSON ও Fetch API **Ch. 11**

Objects, JSON, Async/Await, API থেকে data আনা

🚀 PART 4 — ADVANCED & PROJECTS

12 CSS Animations ও Transitions **Ch. 12**

transition, animation, keyframes, transform

13 Advanced JS — ES6+ ও Modern Web **Ch. 13**

Destructuring, Spread, Modules, LocalStorage, Error handling

14 Projects ও Next Steps **Ch. 14**

৫টি প্রজেক্ট আইডিয়া, Roadmap, কোথায় Deploy করবো

CHAPTER 01 — HTML

🌐 HTML

HTML পরিচিতি ও ওয়েবের কাঠামো

HTML কী, কিভাবে কাজ করে, প্রথম পেজ বানানো

🌐 HTML কী?

HTML (HyperText Markup Language) হলো ওয়েবপেজের **কাঠামো**। এটা কোনো programming language না — এটা একটা **markup language**। ঘরের ইটের দেওয়ালের মতো — যা CSS (রং-রূপ) এবং JavaScript (কার্যকারিতা) ধরে রাখে।

🌐 **HTML**

কাঠামো — *কী আছে*

Heading, Paragraph, Image, Button

🎨 **CSS**

ডিজাইন — *কেমন দেখায়*

Color, Font, Layout, Animation

⚡ **JavaScript**

কার্যকারিতা — *কী করে*

Click, Form submit, Data fetch

📄 HTML ফাইলের কাঠামো

```
<!— প্রতিটা HTML ফাইলের এই কাঠামো লাগে —> <!DOCTYPE html> <!— HTML5 ঘোষণা —> <html lang="bn"> <!— root element —> <head> <!— Browser-এর জন্য তথ্য —> <meta charset="UTF-8"> <!— বাংলা সাপোর্ট —> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>আমার প্রথম পেজ</title> <!— Tab-এ নাম —> <link rel="stylesheet" href="style.css"> <!— CSS file —> </head> <body> <!— যা পেজে দেখা যাবে —> <h1>হ্যালো দুনিয়া! </h1> <p>আমার প্রথম ওয়েবসাইট।</p> <script src="script.js"></script> <!— JS file —> </body> </html>
```

🧱 Tag, Element ও Attribute

```
Opening Tag    Content       Closing Tag
     ↓             ↓               ↓
<h1 class="title">    আমার ওয়েবসাইট    </h1>
      ↑       ↑
 Attribute   Value

সব মিলিয়ে এটাই একটা Element
```

💡 Self-closing Tags — closing tag লাগে না

```
<img src="photo.jpg" alt="ছবি">  <!— ছবি —>  <br>  <!— নতুন লাইন —>
<hr>  <!— আড়াআড়ি রেখা —>  <input type="text">  <!— Input box —>
<meta charset="UTF-8">  <!— Metadata —>
```

🚀 প্রথম পেজ তৈরি করো

1. VS Code খুলো → নতুন ফাইল তৈরি করো → নাম দাও `index.html`
2. টাইপ করো `!` তারপর Tab চাপো → পুরো HTML skeleton তৈরি হয়ে যাবে!
3. Live Server extension ইনস্টল করো → Go Live চাপো → Browser-এ real-time দেখবে

📌 Chapter সারসংক্ষেপ

- 🌐 HTML = ওয়েবপেজের কাঠামো, programming language নয়
- 📄 সব HTML ফাইলে: DOCTYPE → html → head → body
- 🧱 `<tag>` content `</tag>` = Element; `attribute="value"`

- ⚡ VS Code + Live Server = সেরা শুরু

CHAPTER 02 — HTML

🌐 HTML

গুরুত্বপূর্ণ HTML Tags

Heading, Paragraph, Link, Image, List, Table — পেজ বানানোর সব উপকরণ

📄 Text Tags

```
<!— Headings – h1 সবচেয়ে বড়, h6 সবচেয়ে ছোট —> <h1>সবচেয়ে বড় শিরোনাম</h1>
<h2>দ্বিতীয় শিরোনাম</h2> <h3>তৃতীয় শিরোনাম</h3> <!— Paragraph —> <p>এটা একটা
অনুচ্ছেদ।</p> <!— Text formatting —> <strong>মোটা লেখা (Bold)</strong>
<em>বাঁকা লেখা (Italic)</em> <u>আন্ডারলাইন</u> <mark>হাইলাইট</mark> <del>কাটা
লেখা</del> <br> <!— নতুন লাইন —> <hr> <!— আড়াআড়ি রেখা —>
```

🔗 Link ও Image

```
<!— Link —> <a href="https://google.com">Google-এ যাও</a> <a
href="about.html">About পেজ</a> <!— same site —> <a
href="#section1">নিচে যাও</a> <!— same page —> <a href="https://fb.com"
target="_blank">নতুন tab</a> <!— Image —> <img src="photo.jpg"
alt="আমার ছবি" width="300"> <img src="https://picsum.photos/400/300"
alt="Random photo"> <!— Clickable image —> <a
href="https://google.com"> <img src="logo.png" alt="Logo"> </a>
```

📋 List

```
<!— Unordered List (bullet) —> <ul> <li>আম</li> <li>কলা</li>
<li>লিচু</li> </ul> <!— Ordered List (number) —> <ol> <li>প্রথম ধাপ</li>
```

```
<li>দ্বিতীয় ধাপ</li> </ol>
```

📊 Table

```
<table> <thead> <!— Table Header —> <tr> <!— Table Row —> <th>নাম</th>
<!— Header cell —> <th>বয়স</th> <th>শহর</th> </tr> </thead> <tbody>
<tr> <td>রহিম</td> <!— Data cell —> <td>২০</td> <td>ঢাকা</td> </tr> <tr>
<td>করিম</td> <td>২২</td> <td>চট্টগ্রাম</td> </tr> </tbody> </table>
```

📦 div ও span — Container Elements

```
<!— div = block level container (নতুন লাইনে) —> <div class="card">
<h2>Card Title</h2> <p>Card content</p> </div> <!— span = inline
container (একই লাইনে) —> <p>এটা <span style="color:red">লাল</span> রঙের
শব্দ।</p>
```

📌 সারসংক্ষেপ

- 📄 `h1-h6` , `p` , `strong` , `em` — text formatting
- 🔗 `<a href="">` — link, `<img src="" alt="">` — image
- 📋 `ul/ol + li` — list, `table + tr + td` — table
- 📦 `div` — block container, `span` — inline container

CHAPTER 03 — HTML

🌐 HTML

Semantic HTML ও Forms

অর্থপূর্ণ Tags ও ব্যবহারকারীর কাছ থেকে তথ্য নেওয়া

🏗️ Semantic HTML — অর্থপূর্ণ কাঠামো

Semantic tag ব্যবহার করলে browser, search engine এবং screen reader বুঝতে পারে কোন অংশের কী কাজ। শুধু `div` দিলে বোঝা যায় না।

❌ Non-Semantic (এড়িয়ে চলো)

```
<div id="header">...</div>
<div id="nav">...</div> <div
id="content">...</div> <div
id="footer">...</div>
```

✅ Semantic (ব্যবহার করো)

```
<header>...</header> <nav>...
</nav> <main>...</main>
<footer>...</footer>
```

```
<!— সম্পূর্ণ Semantic কাঠামো —> <header> <!— সাইটের উপরের অংশ —> <nav>
<!— Navigation menu —> <ul> <li><a href="/">Home</a></li> <li><a
href="/about">About</a></li> </ul> </nav> </header> <main> <!— পেজের মূল
বিষয়বস্তু —> <article> <!— স্বতন্ত্র article —> <h1>আমার Blog Post</h1>
<section> <!— article-এর অংশ —> <h2>ভূমিকা</h2> <p>এখানে লেখা...</p>
</section> </article> <aside> <!— পাশের তথ্য/sidebar —> <p>Related
links...</p> </aside> </main> <footer> <!— সাইটের নিচের অংশ —> <p>© 2025
আমার ওয়েবসাইট</p> </footer>
```

📝 Forms — ব্যবহারকারীর কাছ থেকে তথ্য নেওয়া

```
<form action="/submit" method="POST"> <!— Text input —> <label for="naam">নাম:</label> <input type="text" id="naam" name="naam" placeholder="তোমার নাম" required> <!— Email —> <label for="email">Email</label> <input type="email" id="email" name="email" required> <!— Password —> <input type="password" placeholder="পাসওয়ার্ড"> <!— Number —> <input type="number" min="1" max="100"> <!— Date —> <input type="date"> <!— Radio buttons —> <input type="radio" name="gender" value="male"> পুরুষ <input type="radio" name="gender" value="female"> মহিলা <!— Checkbox —> <input type="checkbox" name="agree"> শর্তে রাজি <!— Dropdown —> <select name="city"> <option value="dhaka">ঢাকা</option> <option value="ctg">চট্টগ্রাম</option> </select> <!— Textarea —> <textarea name="message" rows="5" placeholder="তোমার বার্তা"></textarea> <!— Submit button —> <button type="submit">Submit</button> </form>
```

📌 সারসংক্ষেপ

- 🏗️ Semantic: header, nav, main, article, section, aside, footer
- 📝 Form: action (কোথায় যাবে), method (GET/POST)
- ⌨️ Input types: text, email, password, number, radio, checkbox, select
- ✅ `required` — ফাঁকা রাখা যাবে না

CHAPTER 04 — CSS

🎨 CSS

CSS পরিচিতি, Selectors ও Box Model

HTML-কে সুন্দর করার জাদু — Selectors, Properties ও Box Model

🎨 CSS কিভাবে লিখবো?

```
/* CSS-এর গঠন */ selector { property: value; property: value; } /* উদাহরণ */ h1 { color: red; font-size: 32px; text-align: center; }
```

🎯 CSS Selectors — কাকে style দেবো

```
/* 1. Element selector — সব h1 */ h1 { color: blue; } /* 2. Class selector — .title class-এর সব element */ .title { font-weight: bold; } /* 3. ID selector — #header id-এর element */ #header { background: navy; } /* 4. Multiple selectors */ h1, h2, h3 { color: darkblue; } /* 5. Descendant selector — div-এর ভেতরের p */ div p { margin: 10px; } /* 6. Pseudo-class — hover করলে */ button:hover { background: green; } a:visited { color: purple; } input:focus { border-color: blue; } /* 7. Pseudo-element */ p::first-line { font-weight: bold; } p::before { content: "→ "; }
```

📦 Box Model — সবকিছুই একটা বাক্স

CSS-এ প্রতিটা element একটা বাক্সের মতো। এই বাক্সের চারটা অংশ আছে:

```
┌──────────────────────────────────────┐
│                margin                │  ← বাইরের ফাঁক
│  ┌──────────────────────────────┐    │
│  │             border           │    │  ← বাক্সের সীমানা
│  │  ┌────────────────────────┐  │    │
│  │  │          padding       │  │    │  ← ভেতরের ফাঁক
│  │  │  ┌──────────────────┐  │  │    │
│  │  │  │       content    │  │  │    │  ← আসল বিষয়বস্তু
│  │  │  │  (text, image, etc)  │  │    │
│  │  │  └──────────────────┘  │  │    │
│  │  └────────────────────────┘  │    │
│  └──────────────────────────────┘    │
└──────────────────────────────────────┘
```

```
.card { /* Content size */ width: 300px; height: 200px; /* Padding — ভেতরের ফাঁক */ padding: 20px; /* চারদিকে */ padding: 10px 20px; /* উপর-নিচ, বাম-ডান */ padding-top: 15px; /* শুধু উপরে */ /* Margin — বাইরের ফাঁক */ margin: 20px; /* চারদিকে */ margin: 0 auto; /* মাঝখানে রাখো */ /* Border — সীমানা */ border: 2px solid #333; border-radius: 10px; /* গোল কোণ */ /* box-sizing: border-box — padding/border size-এ যোগ হয় না */ box-sizing: border-box; /* সবসময় এটা ব্যবহার করো! */ }
```

📍 Display ও Position

```
/* Display */ .element { display: block; /* নতুন লাইনে, পুরো width */ display: inline; /* একই লাইনে, content width */ display: inline-block; /* একই লাইনে কিন্তু size দেওয়া যায় */ display: none; /* লুকিয়ে যায় */ display: flex; /* Flexbox (পরের chapter!) */ display: grid; /* Grid (পরের chapter!) */ } /* Position */ .box { position: static; /* default — flow-তে থাকে */ position: relative; /* নিজের জায়গা থেকে সরে */ position: absolute; /* parent-এর সাপেক্ষে */ position: fixed; /* screen-এ আটকা (navbar!) */ position: sticky; /* scroll করলে আটকে যায় */ top: 20px; left: 30px; }
```

📌 সারসংক্ষেপ

- 🎯 Selector: element, .class, #id, :hover, ::before
- 📦 Box Model: content → padding → border → margin
- ✅ `box-sizing: border-box` — সবসময় ব্যবহার করো
- 📍 Position: static, relative, absolute, fixed, sticky

CHAPTER 05 — CSS

🎨 CSS

Typography, Colors ও CSS Variables

Font, Color, Gradient ও CSS Custom Properties দিয়ে পেশাদার ডিজাইন

✍️ Typography — লেখার সৌন্দর্য

```
body { /* Google Fonts ব্যবহার */ font-family: 'Hind Siliguri', sans-serif;
font-size: 16px; /* base size */ line-height: 1.6; /* লাইনের ফাঁক */ } h1
{ font-size: 2.5rem; /* base-এর 2.5 গুণ */ font-weight: 700; /* bold */
letter-spacing: -0.02em; /* অক্ষরের ফাঁক */ text-transform: uppercase;/* বড়
হাতে */ text-decoration: none; /* underline সরাও */ text-align: center; /*
মাঝখানে */ text-shadow: 2px 2px 4px rgba(0,0,0,0.3); /* ছায়া */ }
```

🎨 Colors — রঙের উপায়

```
.element { /* Named colors */ color: red; color: coral; /* Hex (সবচেয়ে
বেশি ব্যবহার) */ color: #e63946; color: #fff; /* shorthand */ /* RGB */
color: rgb(230, 57, 70); /* RGBA — a = transparency (0-1) */ color:
rgba(230, 57, 70, 0.5); /* 50% স্বচ্ছ */ background: rgba(0, 0, 0, 0.8); /*
dark overlay */ /* HSL */ color: hsl(350, 78%, 56%); } /* Gradient */
.hero { background: linear-gradient(135deg, #667eea, #764ba2);
background: radial-gradient(circle, #667eea, #764ba2); }
```

🔧 CSS Variables — পুনরায় ব্যবহারযোগ্য মান

CSS Variables (Custom Properties) দিয়ে একটা জায়গায় রঙ বা মান সংরক্ষণ করো, সব জায়গায় ব্যবহার করো। পরে বদলাতে হলে একটা জায়গায় বদলালেই হয়।

```
/* :root-এ define করো — সব পেজে available */ :root { --primary-color: #2563eb; --secondary-color: #7c3aed; --text-color: #1e293b; --bg-color: #f8fafc; --font-size-base: 16px; --border-radius: 8px; --shadow: 0 4px 6px rgba(0,0,0,0.1); } /* var() দিয়ে ব্যবহার করো */ .button { background: var(--primary-color); color: white; border-radius: var(--border-radius); box-shadow: var(--shadow); } .card { background: var(--bg-color); color: var(--text-color); border-radius: var(--border-radius); }
```

🌑 Dark Mode — CSS Variables দিয়ে

```
:root { --bg: #ffffff; --text: #000000; } [data-theme="dark"] { --bg: #0f172a; --text: #e2e8f0; } body { background: var(--bg); color: var(--text); transition: background 0.3s, color 0.3s; }
```

📌 সারসংক্ষেপ

- ✍️ Typography: font-family, font-size, line-height, font-weight
- 🎨 Color: hex (#fff), rgba(0,0,0,0.5), hsl()
- 🌈 Gradient: linear-gradient(), radial-gradient()
- 🔗 `--variable: value` define, `var(--variable)` ব্যবহার

CHAPTER 06 — CSS

🎨 CSS

🧲 Flexbox — আধুনিক Layout

Elements সাজানোর সবচেয়ে সহজ এবং শক্তিশালী উপায়

🧲 Flexbox কী?

Flexbox একটা CSS Layout system। Parent-এ `display: flex` দিলে তার সব সরাসরি children flex items হয়ে যায় এবং সহজে সাজানো যায়।

```
/* Flex Container (Parent) */ .container { display: flex; /* Direction — কোন দিকে সাজাবো */ flex-direction: row; /* → বাম থেকে ডান (default) */ flex-direction: row-reverse; /* ← ডান থেকে বাম */ flex-direction: column; /* ↓ উপর থেকে নিচ */ flex-direction: column-reverse; /* ↑ নিচ থেকে উপর */ /* Main axis alignment (row হলে horizontal) */ justify-content: flex-start; /* বাম থেকে */ justify-content: flex-end; /* ডান থেকে */ justify-content: center; /* মাঝখানে */ justify-content: space-between; /* দুপাশে আটকে, মাঝে ফাঁক */ justify-content: space-around; /* সব দিকে সমান ফাঁক */ justify-content: space-evenly; /* একেবারে সমান */ /* Cross axis alignment (row হলে vertical) */ align-items: stretch; /* পূর্ণ height (default) */ align-items: flex-start; /* উপরে */ align-items: flex-end; /* নিচে */ align-items: center; /* মাঝখানে ✨ */ /* Wrap — ছোট হলে পরের লাইনে যাবে */ flex-wrap: wrap; gap: 20px; /* items-এর মধ্যে ফাঁক */ }
```

⭐ সবচেয়ে কাজের Flexbox trick

```
/* Perfect center করা — horizontal ও vertical */ .center-everything {
display: flex; justify-content: center; align-items: center; min-height:
```

```
100vh; /* পুরো screen উচ্চতা */ }
```

```
/* Navbar layout */ .navbar { display: flex; justify-content: space-between; align-items: center; padding: 16px 32px; background: #0f172a; } /* Card grid layout */ .card-grid { display: flex; flex-wrap: wrap; gap: 24px; } .card-grid .card { flex: 1 1 300px; /* min 300px, বাকি space নাও */ }
```

🔢 Flex Item Properties

```
/* Flex Item (Children) */ .item { flex: 1; /* available space সমানভাবে নাও */ flex: 2; /* অন্যদের দ্বিগুণ space নাও */ flex-grow: 1; /* বাড়তে পারবে */ flex-shrink: 0; /* ছোট হবে না */ flex-basis: 200px;/* base size */ order: -1; /* HTML-এর আগে দেখাবে */ align-self: center; /* শুধু এই item আলাদাভাবে */ }
```

📌 সারসংক্ষেপ

- 🧲 `display: flex` — parent-এ দিলে flex শুরু
- ↔️ `justify-content` — main axis-এ সাজানো
- ↕️ `align-items` — cross axis-এ সাজানো
- ⭐ Perfect center: flex + justify-content:center + align-items:center
- 📐 `gap` — items-এর মধ্যে ফাঁক

CHAPTER 07 — CSS

🎨 CSS

📐 CSS Grid ও Responsive Design

Two-dimensional layout ও সব device-এ সুন্দর দেখানো

📐 CSS Grid — দুই দিকে Layout

Flexbox এক দিকে (row বা column) কাজ করে। Grid দুই দিকে (row এবং column) কাজ করে — magazine layout-এর জন্য পারফেক্ট।

```
.grid-container { display: grid; /* 3 column grid */ grid-template-columns: 200px 200px 200px; /* fixed */ grid-template-columns: 1fr 1fr 1fr; /* fraction */ grid-template-columns: repeat(3, 1fr); /* shorthand */ grid-template-columns: repeat(auto-fit, minmax(250px, 1fr)); /* responsive! */ /* Rows */ grid-template-rows: 100px auto 60px; /* Gap */ gap: 24px; row-gap: 20px; column-gap: 16px; } /* Item placement */ .header { grid-column: 1 / -1; } /* সব column জুড়ে */ .sidebar { grid-row: 2 / 4; } /* 2 row জুড়ে */
```

```
/* সবচেয়ে দরকারি: Responsive card grid */ .card-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(280px, 1fr)); gap: 24px; } /* এটা automatically: mobile-এ 1 column, tablet-এ 2, desktop-এ 3+ দেখাবে! */
```

📱 Responsive Design — সব device-এ সুন্দর

Media Query ব্যবহার করে screen-এর size অনুযায়ী আলাদা CSS লেখা যায়।

```
/* Mobile-first approach — ছোট থেকে বড় */ /* Mobile (default — কোনো query নেই) */ .container { padding: 16px; } .nav-links { display: none; } /* mobile-এ hidden */ /* Tablet (768px+) */ @media (min-width: 768px) { .container { padding: 24px; } .nav-links { display: flex; } /* tablet-এ দেখায় */ .grid { grid-template-columns: repeat(2, 1fr); } } /* Desktop (1024px+) */ @media (min-width: 1024px) { .container { max-width: 1200px; margin: 0 auto; } .grid { grid-template-columns: repeat(3, 1fr); } } /* Dark mode */ @media (prefers-color-scheme: dark) { body { background: #0f172a; color: #e2e8f0; } }
```

💡 Viewport Units

```
/* vw = viewport width, vh = viewport height */ .hero { height: 100vh; } /* পুরো screen উচ্চতা */ .title { font-size: 5vw; } /* screen width-এর 5% */ .sidebar { width: 30vw; } /* screen width-এর 30% */
```

📌 সারসংক্ষেপ

- 📐 `display: grid` + `grid-template-columns` — 2D layout
- ⭐ `repeat(auto-fit, minmax(280px, 1fr))` — auto-responsive grid
- 📱 `@media (min-width: 768px)` — responsive CSS
- 📐 Mobile-first: ছোট screen-এর CSS আগে লেখো

CHAPTER 08 — JAVASCRIPT

JavaScript

JavaScript পরিচিতি ও Basics

Variables, Data Types, Conditions, Loops — ওয়েবকে জীবন্ত করা

⚡ JavaScript কীভাবে যোগ করবো?

```
/* HTML-এ যোগ করার উপায় */ <!— 1. External file (সবচেয়ে ভালো) —>
<script src="script.js" defer></script> <— body-র শেষে <!— 2. Inline —>
<script> console.log("Hello!"); // browser console-এ দেখায় </script>
```

📦 Variables — ডেটা সংরক্ষণ

```
// 3 ধরনের variable declaration let naam = "Rahim"; // পরিবর্তন করা যায় ✅
const PI = 3.14; // পরিবর্তন করা যায় না ❌ var boyos = 20; // পুরনো পদ্ধতি,
এড়িয়ে চলো // Data Types let text = "Hello"; // String let number = 42; //
Number let decimal = 3.14; // Number (float) let isTrue = true; //
Boolean let nothing = null; // Null let undef; // Undefined let list =
[1, 2, 3]; // Array let person = { naam: "Rahim" }; // Object
```

🔀 Conditions ও Loops

```
// if/else let boyos = 20; if (boyos ≥ 18) { console.log("প্রাপ্তবয়স্ক"); }
else if (boyos ≥ 13) { console.log("কিশোর"); } else {
console.log("শিশু"); } // Ternary operator — এক লাইনে if/else let status =
boyos ≥ 18 ? "Adult" : "Minor"; // for loop for (let i = 0; i < 5; i++)
{ console.log(`সংখ্যা: ${i}`); // template literal } // Array-তে loop const
```

```
fruits = ["আম", "কলা", "লিচু"]; for (const fruit of fruits) {
console.log(fruit); } // while loop let count = 0; while (count < 3) {
console.log(count); count++; }
```

🔧 Template Literals ও String

```
const naam = "Rahim"; const boyos = 20; // পুরনো পদ্ধতি (এড়িয়ে চলো)
console.log("নাম: " + naam + ", বয়স: " + boyos); // Template Literal
(সবসময় এটা ব্যবহার করো!) console.log(`নাম: ${naam}, বয়স: ${boyos}`);
console.log(`গণনা: ${boyos * 2}`); // expression-ও চলে // Multi-line
string const html = ` <div class="card"> <h2>${naam}</h2> <p>বয়স:
${boyos}</p> </div> `;
```

📌 সারসংক্ষেপ

- 📦 `let` — পরিবর্তনযোগ্য, `const` — স্থির
- 🔀 `if/else if/else` — শর্ত, Ternary: `? :`
- 🔁 `for`, `for...of`, `while` — loop
- ✨ `` `Hello ${name}` `` — Template Literal, সবসময় এটা ব্যবহার করো

CHAPTER 09 — JAVASCRIPT

JavaScript

Functions ও Arrays

কোড পুনরায় ব্যবহার ও ডেটার তালিকা নিয়ে কাজ

⚙️ Functions

```
// 1. Function Declaration function greet(naam) { return `হ্যালো, ${naam}!`; } console.log(greet("Rahim")); // 2. Arrow Function (modern) — ← সবচেয়ে বেশি ব্যবহার করো const greet = (naam) => `হ্যালো, ${naam}!`; // Multiple parameters const add = (a, b) => a + b; console.log(add(10, 20)); // 30 // Default parameter const greetUser = (naam = "বন্ধু") => `হ্যালো, ${naam}!`; console.log(greetUser()); // হ্যালো, বন্ধু! // Higher-order function — function-কে parameter হিসেবে দেওয়া const doTwice = (fn, value) => fn(fn(value)); console.log(doTwice(x => x * 2, 5)); // 20
```

📋 Arrays ও দরকারি Methods

```
const fruits = ["আম", "কলা", "লিচু"]; // Access console.log(fruits[0]); // আম console.log(fruits.length); // 3 // Add/Remove fruits.push("আনারস"); // শেষে যোগ fruits.unshift("জাম"); // শুরুতে যোগ fruits.pop(); // শেষেরটা বাদ fruits.splice(1, 1); // index 1 থেকে 1টা বাদ // Powerful Methods const numbers = [1, 2, 3, 4, 5]; // map — প্রতিটা item transform করো const doubled = numbers.map(n => n * 2); console.log(doubled); // [2, 4, 6, 8, 10] // filter — শর্ত মেলে এমনগুলো রাখো const evens = numbers.filter(n => n % 2 === 0); console.log(evens); // [2, 4] // find — প্রথম match খোঁজো const found = numbers.find(n => n > 3); console.log(found); // 4 // reduce — সব মিলিয়ে একটা ফলাফল const sum = numbers.reduce((total, n) => total + n, 0); console.log(sum); // 15 //
```

```
includes — আছে কিনা console.log(numbers.includes(3)); // true // forEach
— প্রতিটায় কাজ করো (map-এর মতো, কিন্তু return নেই) fruits.forEach(fruit ⇒
console.log(fruit)); // sort — সাজানো const sorted = [3, 1, 4,
1].sort((a, b) ⇒ a - b); console.log(sorted); // [1, 1, 3, 4]
```

📌 সারসংক্ষেপ

- ⚙️ Arrow function: `const fn = (x) ⇒ x * 2`
- 📋 `push/pop/splice` — array modify করা
- 🗺️ `map()` — transform, `filter()` — বাছাই, `reduce()` — একটায় পরিণত
- 🔍 `find()` — খোঁজা, `includes()` — আছে কিনা

CHAPTER 10 — JAVASCRIPT

JavaScript

🌳 DOM Manipulation

JavaScript দিয়ে HTML page বদলানো — ওয়েবের সবচেয়ে শক্তিশালী দিক

🌳 DOM কী?

DOM (Document Object Model) হলো HTML document-এর JavaScript representation। JavaScript দিয়ে DOM-এর যেকোনো element খুঁজে পাওয়া, বদলানো, যোগ বা বাদ দেওয়া যায়।

🔍 Elements খোঁজা

```
// querySelector — CSS selector দিয়ে প্রথমটা const title =
document.querySelector('h1'); const btn = document.querySelector('.btn');
const header = document.querySelector('#header'); // querySelectorAll —
সব মিলিয়ে NodeList const allCards = document.querySelectorAll('.card');
const allImages = document.querySelectorAll('img'); // Loop করে সব নিয়ে
কাজ করো allCards.forEach(card => { card.style.border = "2px solid blue";
});
```

✏️ Content ও Style বদলানো

```
const elem = document.querySelector('#title'); // Text বদলানো
elem.textContent = "নতুন শিরোনাম"; // plain text elem.innerHTML = "
<strong>Bold</strong>"; // HTML support // Style বদলানো elem.style.color
= "red"; elem.style.fontSize = "24px"; elem.style.display = "none"; //
লুকিয়ে দাও // Attribute বদলানো const img = document.querySelector('img');
img.setAttribute('src', 'new-photo.jpg'); img.setAttribute('alt', 'নতুন
```

```
ছবি'); // CSS class যোগ/বাদ/toggle elem.classList.add('active'); // class
যোগ elem.classList.remove('hidden'); // class বাদ
elem.classList.toggle('dark'); // থাকলে বাদ, না থাকলে যোগ
elem.classList.contains('active'); // আছে কিনা: true/false
```

🖱️ Events — কিছু হলে কাজ করো

```
const btn = document.querySelector('#myBtn'); // Click event
btn.addEventListener('click', () => { console.log("Button চাপা হয়েছে!");
}); // Form submit const form = document.querySelector('form');
form.addEventListener('submit', (event) => { event.preventDefault(); //
page reload বন্ধ const naam = document.querySelector('#naam').value;
console.log(`নাম: ${naam}`); }); // Input change const input =
document.querySelector('input'); input.addEventListener('input', (e) => {
console.log(`লিখছো: ${e.target.value}`); }); // Common events // click,
dblclick, mouseover, mouseout // keydown, keyup, keypress // submit,
input, change // load, resize, scroll
```

➕ Element তৈরি ও যোগ করা

```
// নতুন element তৈরি করো const newCard = document.createElement('div');
newCard.classList.add('card'); newCard.innerHTML = ` <h3>নতুন Card</h3>
<p>এটা JavaScript দিয়ে তৈরি</p> `; // Page-এ যোগ করো const container =
document.querySelector('.container'); container.appendChild(newCard); //
শেষে যোগ container.prepend(newCard); // শুরুতে যোগ // Remove করো
newCard.remove();
```

📌 সারসংক্ষেপ

- 🔍 `querySelector('.class')` — element খোঁজো
- ✏️ `.textContent` , `.innerHTML` , `.style.color` — বদলাও
- 🎨 `.classList.add/remove/toggle` — CSS class

- 🖱 `addEventListener('click', fn)` — event শোনো
- ➕ `createElement + appendChild` — নতুন element তৈরি

CHAPTER 11 — JAVASCRIPT

JavaScript

🌐 Objects, JSON ও Fetch API

Objects, JSON, Async/Await ও API থেকে real data আনা

📦 Objects — Key-Value জোড়া

```
// Object তৈরি const student = { naam: "Rahim", boyos: 20, shahor: "Dhaka", hobbies: ["coding", "reading"], address: { // nested object road: "Mirpur 10", city: "Dhaka" }, greet() { // method return `হ্যালো, আমি ${this.naam}`; } }; // Access console.log(student.naam); // dot notation console.log(student['boyos']); // bracket notation console.log(student.address.city); // nested console.log(student.greet()); // method call // Destructuring — সহজে বের করা const { naam, boyos, shahor } = student; console.log(naam, boyos); // Rahim 20 // Spread — copy করা const updatedStudent = { ...student, boyos: 21 };
```

📄 JSON — ডেটা আদান-প্রদানের ভাষা

```
// JSON = JavaScript Object Notation // API থেকে এই format-এ ডেটা আসে const jsonString = '{"naam": "Rahim", "boyos": 20}'; // JSON String → JavaScript Object const obj = JSON.parse(jsonString); console.log(obj.naam); // Rahim // JavaScript Object → JSON String const str = JSON.stringify(obj); console.log(str); // {"naam":"Rahim","boyos":20} // Pretty print console.log(JSON.stringify(obj, null, 2));
```

🌐 Fetch API — ইন্টারনেট থেকে ডেটা আনা

```
// Async/Await দিয়ে API call (সবচেয়ে আধুনিক পদ্ধতি) const getUsers = async () => { try { // API-তে request পাঠাও const response = await fetch('https://jsonplaceholder.typicode.com/users'); // JSON-এ convert করো const users = await response.json(); // ব্যবহার করো users.forEach(user => { console.log(`${user.name}: ${user.email}`); }); } catch (error) { console.error('Error:', error); } }; getUsers();
```

```
// Real-world example: Users দেখানো const showUsers = async () => { const container = document.querySelector('#users'); container.innerHTML = 'Loading...'; const res = await fetch('https://jsonplaceholder.typicode.com/users'); const users = await res.json(); container.innerHTML = users.map(user => ` <div class="card"> <h3>${user.name}</h3> <p>📧 ${user.email}</p> <p>🌐 ${user.website}</p> </div> `).join(''); }; showUsers();
```

📌 সারসংক্ষেপ

- 📦 Object: key-value pair, nested object, method
- ✂️ Destructuring: `const { naam, boyos } = student`
- 📄 `JSON.parse()` → Object, `JSON.stringify()` → String
- 🌐 `async/await + fetch()` — API থেকে ডেটা আনা

CHAPTER 12 — ADVANCED

🚀 Advanced

✨ CSS Animations ও Transitions

Motion দিয়ে ওয়েবসাইটকে প্রাণবন্ত করা

🔄 Transitions — মসৃণ পরিবর্তন

```
/* transition: property duration timing-function delay */ .button {
background: #2563eb; color: white; padding: 12px 24px; border-radius:
8px; transition: all 0.3s ease; /* সব property, 0.3s, smooth */ }
.button:hover { background: #1d4ed8; transform: translateY(-2px); /* উপরে
উঠে */ box-shadow: 0 8px 24px rgba(37,99,235,0.4); } /* Individual
transitions */ .card { transition: transform 0.3s ease, box-shadow 0.3s
ease, opacity 0.2s linear; } .card:hover { transform: scale(1.03); box-
shadow: 0 20px 40px rgba(0,0,0,0.15); }
```

🎬 Transform — Element-কে রূপান্তর

```
.element { transform: translateX(50px); /* ডানে সরাও */ transform:
translateY(-20px); /* উপরে সরাও */ transform: scale(1.5); /* দেড়গুণ বড় */
transform: rotate(45deg); /* ৪৫ ডিগ্রি ঘোরাও */ transform: skew(10deg); /*
বাঁকাও */ /* একসাথে */ transform: translateY(-4px) scale(1.05); }
```

🎞 Keyframe Animations — পূর্ণ Animation

```
/* @keyframes দিয়ে animation define করো */ @keyframes fadeIn { from {
opacity: 0; transform: translateY(20px); } to { opacity: 1; transform:
```

```
translateY(0); } } @keyframes spin { from { transform: rotate(0deg); } to
{ transform: rotate(360deg); } } @keyframes pulse { 0%, 100% { transform:
scale(1); } 50% { transform: scale(1.08); } } /* animation: name duration
timing-function delay iteration-count */ .card { animation: fadeIn 0.5s
ease forwards; } .loader { animation: spin 1s linear infinite; } .button
{ animation: pulse 2s ease-in-out infinite; } /* Staggered animation —
একটার পর একটা */ .card:nth-child(1) { animation-delay: 0s; } .card:nth-
child(2) { animation-delay: 0.1s; } .card:nth-child(3) { animation-delay:
0.2s; }
```

📌 সারসংক্ষেপ

- 🔄 `transition: all 0.3s ease` — hover-এ মসৃণ পরিবর্তন
- 🔧 `transform: scale/translate/rotate` — রূপান্তর
- 🎞 `@keyframes + animation` — পূর্ণ animation
- ⏱ `animation-delay` — staggered effect তৈরি

CHAPTER 13 — ADVANCED

🚀 Advanced

⚡ Advanced JavaScript — ES6+

Destructuring, Spread, Modules, LocalStorage, Error handling

✂️ Destructuring ও Spread

```
// Array Destructuring const [first, second, ...rest] = [1, 2, 3, 4, 5];
console.log(first, second, rest); // 1, 2, [3,4,5] // Object
Destructuring const { naam, boyos, ...otherInfo } = student; // Spread
operator — ছড়িয়ে দাও const arr1 = [1, 2, 3]; const arr2 = [4, 5, 6]; const
combined = [...arr1, ...arr2]; // [1,2,3,4,5,6] const obj1 = { a: 1, b: 2
}; const obj2 = { c: 3 }; const merged = { ...obj1, ...obj2 }; // {a:1,
b:2, c:3} // Optional chaining — safely nested property access const city
= user?.address?.city; // error ছাড়াই // Nullish coalescing —
null/undefined হলে default দাও const name = user.naam ?? "অপরিচিত";
```

💾 LocalStorage — Browser-এ ডেটা সংরক্ষণ

```
// LocalStorage — browser বন্ধ করলেও থাকে // সংরক্ষণ করো (শুধু string)
localStorage.setItem('naam', 'Rahim'); // Object সংরক্ষণ করতে
JSON.stringify const user = { naam: "Rahim", boyos: 20 };
localStorage.setItem('user', JSON.stringify(user)); // পড়ো const
savedName = localStorage.getItem('naam'); const savedUser =
JSON.parse(localStorage.getItem('user')); // বাদ দাও
localStorage.removeItem('naam'); localStorage.clear(); // Use case: Dark
mode সংরক্ষণ const toggleDark = () ⇒ {
document.body.classList.toggle('dark'); const isDark =
```

```
document.body.classList.contains('dark');
localStorage.setItem('darkMode', isDark); };
```

🛡️ Error Handling ও Try/Catch

```
// try/catch দিয়ে error সামলাও const fetchData = async (url) => { try {
const response = await fetch(url); if (!response.ok) { throw new
Error(`HTTP Error: ${response.status}`); } const data = await
response.json(); return data; } catch (error) { console.error('Error:',
error.message); // UI-তে error দেখাও
document.querySelector('#error').textContent = error.message; } finally {
// সবসময় চলে — loading hide করো
document.querySelector('.loader').style.display = 'none'; } };
```

📌 সারসংক্ষেপ

- ✂️ Destructuring: array ও object থেকে সহজে বের করা
- 🔀 Spread `...` : array/object copy ও merge
- 💾 `localStorage` — browser-এ স্থায়ীভাবে সংরক্ষণ
- 🛡️ `try/catch/finally` — error সামলানো

CHAPTER 14 — PROJECTS

🚀 Projects

🏗️ Projects ও Next Steps

৫টি প্রজেক্ট আইডিয়া, Roadmap এবং কোথায় Deploy করবো

🏗️ ৫টি Project — শুরু থেকে Advanced

🟢 Project 1: Personal Portfolio

নিজের পরিচয় সাইট। HTML + CSS শেখার পরেই বানাও।

HTML CSS Flexbox

নাম, কাজ, ছবি, Contact form

🟡 Project 2: Todo App

কাজের তালিকা। JS শেখার পরেই বানাও।

JS DOM localStorage Events

Add, delete, complete, save করা

🟠 Project 3: Weather App

Real-time আবহাওয়া। Fetch API শেখার পরে।

Fetch API JSON OpenWeather API

শহরের নাম দাও, আবহাওয়া দেখাও

🔵 Project 4: Quiz App

MCQ Quiz। Arrays ও DOM-এর ভালো practice।

Arrays DOM Timer

প্রশ্ন, option, score, timer

🔴 Project 5: E-commerce Product Page

পণ্যের পেজ, cart, filter। সব কিছু একসাথে।

HTML CSS Grid JS Fetch API localStorage

📅 Roadmap — এরপর কোথায় যাবো?

1 🌱 এখন (এই গাইড শেষ)

HTML Semantic CSS Flexbox CSS Grid JS DOM Fetch API

2 ⚡ পরবর্তী ধাপ

Git & GitHub Tailwind CSS React JS Node.js Basics

3 🚀 Advanced

Next.js TypeScript Node + Express MongoDB / SQL REST API বানানো

🚀 Deploy — Live করো!

Platform	কী সুবিধা	দাম
GitHub Pages	Static site — সবচেয়ে সহজ	Free
Netlify	Drag & drop deploy, custom domain	Free
Vercel	React/Next.js-এর জন্য সেরা	Free
Cloudflare Pages	দ্রুত, unlimited bandwidth	Free

📚 দরকারি Resources

📖 শেখার সাইট

- **MDN Web Docs** — developer.mozilla.org

🛠 দরকারি Tools

- **VS Code** — সেরা editor
- **Chrome DevTools** — F12 চাপো

- **W3Schools** — w3schools.com
- **CSS-Tricks** — css-tricks.com
- **JavaScript.info** — javascript.info
- **freeCodeCamp** — freecodecamp.org

- **Figma** — design দেখো
- **Coolors.co** — color palette
- **Google Fonts** — fonts
- **Canva** — design resource

🌐 "Every website you've ever used was built by someone who once knew nothing about web development."

প্রতিটা কোড নিজে লিখো, ভুল করো, Google করো, আবার চেষ্টা করো। এটাই শেখার একমাত্র উপায়।

Web Development Guide (HTML + CSS + JS) — v1.0 | 14 Chapters | Zero to Advanced